

Classroom B - C Lab Worksheet (Single File)

Student Task

Build **one single C file** named `week7_lab_b.c`.

Rules:

1. Use only one `#include <stdio.h>`.
2. Put each question in a separate function.
3. Write only one `main()` and call all question functions inside `main()` in order.
4. Questions 1-5 are normal coding tasks.
5. Questions 6-10 start from intentionally broken code. In each of those questions, fix the broken function before testing it.
6. Everything needed to solve the worksheet is described here. Do not assume any extra starter code.
7. Do not use `<string.h>` functions such as `strlen`, `strcpy`, `strcat`, or `strcmp`. The purpose is to build these ideas manually.

Implementation notes:

- You may reuse helper functions across questions.
- Define helper functions before the question functions that call them, or write function prototypes before `main`.
- Do not hard-code the expected output. Your output must come from your functions.
- For functions that return a yes/no result, return 1 for yes/true and 0 for no/false.

Total: 100 points Each question is worth **10 points**.

Suggested `main()`:

```
int main(void) {
    pointer_string_printer();
    character_counter();
    safe_string_copy();
    longest_shortest_month();
    uppercase_converter();
    broken_str_cat_p();
    broken_str_cpy_rev_p();
    broken_is_palindrome_p();
    broken_str_starts_with_p();
    broken_last_char_p();
    return 0;
}
```

Question 1 - Pointer String Printer [10 points]

Write `print_string_p` that prints a string character by character using pointer traversal, and `str_len_p` that returns the string length.

Requirements:

1. Write `print_string_p(char *p)` using a for loop that advances the pointer until it reaches `'\0'`. Print each character with `printf("%c", *p)`.
2. Write `str_len_p(char *p)` using two pointers. Keep the original pointer `p` unchanged, use a second pointer `q` to scan forward until `*q == '\0'`, then return `q - p`.
3. Inside `pointer_string_printer`, create:
 - `char s1[20] = "Systems";`
 - `char s2[20] = "Programming";`
4. For each string: print it using `print_string_p`, then print its length using `str_len_p`.

Expected output:

```
Systems (length: 7)
Programming (length: 11)
```

Question 2 - Character Counter [10 points]

Write a function that counts how many times a given character appears in a string using pointer traversal.

Requirements:

1. Write `char_count_p(char *s, char c)` that traverses the string with a pointer and counts occurrences of `c`.
2. Initialize a count variable to 0.
3. Use a while loop: while `*s` is not `'\0'`, check if `*s == c`, then advance the pointer.
4. Inside `character_counter`, test with `char str[30] = "programming"`:
 - Count `'g'` -> expected: 2
 - Count `'r'` -> expected: 2
 - Count `'z'` -> expected: 0

Expected output:

```
'g' in "programming": 2
'r' in "programming": 2
'z' in "programming": 0
```

Question 3 - Safe String Copy [10 points]

Demonstrate that `str_cpy_p` creates an independent copy of a string.

Requirements:

1. Write `str_cpy_p(char *dest, const char *src)` using pointers. Copy each character from `src` to `dest` until `src` reaches `'\0'`, then write one `'\0'` at the end of `dest`.

2. The function must not print anything. Its only job is to fill the destination array.
3. Inside `safe_string_copy`, create `char original[20] = "hello"` and `char copy[20]`.
4. Copy `original` into `copy` using `str_cpy_p`.
5. Modify only the copy: change `copy[0]` to `'H'`.
6. Print both strings after the modification.

Goal:

- Show that modifying `copy` does not affect `original`.

Expected output:

```
original: "hello"
copy:     "Hello"
```

Question 4 - Longest and Shortest Month [10 points]

Find the longest and shortest month names from a 2D character array using `str_len_p`.

Requirements:

1. Inside `longest_shortest_month`, create `char month[12][10]` with all 12 month names.
2. Use `str_len_p` to calculate the length of each month name.
3. Find and print the longest month name and its length.
4. Find and print the shortest month name and its length.
5. For both searches, start with the first month as the current best. Then scan the remaining months one by one. If a month has a longer length than the current longest, update the longest. If a month has a shorter length than the current shortest, update the shortest.

Expected output:

```
Longest month:  September (9)
Shortest month:  May (3)
```

Question 5 - Uppercase Converter [10 points]

Write a function that converts all lowercase letters in a string to uppercase, in place, using pointer traversal.

Requirements:

1. Write `to_upper_p(char *s)` that traverses the string with a pointer.
2. For each character, check if it is a lowercase letter: `*s >= 'a' && *s <= 'z'`.
3. If it is lowercase, convert it to uppercase by subtracting 32: `*s = *s - 32`.
4. Leave all other characters (uppercase letters, digits, spaces, punctuation) unchanged.

5. Inside `uppercase_converter`, test with `char str[30] = "Hello World 123"`.

6. Print the string before and after the conversion.

Hint: In ASCII, `'a' - 'A' = 32`. So `'a' - 32 = 'A'`, `'b' - 32 = 'B'`, etc.

Expected output:

```
Before: "Hello World 123"
After:  "HELLO WORLD 123"
```

Question 6 - Broken Code: `str_cat_p` [10 points]

The function below is supposed to append `src` to the end of `dest` using pointers. It has **one bug** that causes the destination string to remain unchanged after the call.

```
void str_cat_p(char *dest, const char *src) {
    while( *dest ) dest++;
    while( *src ) {
        *dest = *src;
        src++;
    }
    *dest = '\0';
}
```

Fix `str_cat_p` and test it inside `broken_str_cat_p` with:

```
char s1[30] = "Good";
char s2[10] = " Morning";
str_cat_p( s1, s2 );
printf( "%s\n", s1 );
```

Expected output after fix:

```
Good Morning
```

Hint:

- After copying a character from `src` into `dest`, which pointer(s) should move forward? If `dest` does not advance, every character from `src` is written to the same position and then overwritten by `'\0'`.
-

Question 7 - Broken Code: `str_cpy_rev_p` [10 points]

The function below is supposed to copy a string in reverse order. It has **one bug** that causes the destination string to appear empty when printed.

```
void str_cpy_rev_p(char *dest, char *src) {
    char *p = src;
    while( *p ) p++;
    while( p >= src ) {
        *dest = *p;
        p--;
        dest++;
    }
}
```

```

    *dest = '\0';
}

```

Fix `str_cpy_rev_p` and test it inside `broken_str_cpy_rev_p` with:

```

char s1[20] = "abcde";
char s2[20] = "";
str_cpy_rev_p( s2, s1 );
printf( "s1: \"%s\"\n", s1 );
printf( "s2: \"%s\"\n", s2 );

```

Expected output after fix:

```

s1: "abcde"
s2: "edcba"

```

Hint:

- After the first `while` loop, `p` points to the `'\0'` character. What is the very first character written into `dest`?
- The first copied character should be the last real character of `src`, not `'\0'`. Decide whether to move `p` back before copying or at the beginning of each copy step.

Question 8 - Broken Code: `is_palindrome_p` [10 points]

The function below is supposed to check if a string is a palindrome using two pointers. It has **two bugs**.

```

int is_palindrome_p(char *s) {
    char *left = s;
    char *right = s;
    while( *right ) right++;
    while( left < right ) {
        if( *left == *right ) return 0;
        left++;
        right--;
    }
    return 1;
}

```

Fix `is_palindrome_p` and test it inside `broken_is_palindrome_p` with:

```

printf( "madam:  %d\n", is_palindrome_p( "madam" ) );
printf( "hello:  %d\n", is_palindrome_p( "hello" ) );
printf( "racecar: %d\n", is_palindrome_p( "racecar" ) );

```

Expected output after fix:

```

madam:  1
hello:  0
racecar: 1

```

Hint:

- After the first `while` loop, `right` points to `'\0'`. Should the comparison start from `'\0'` or from the last real character?
- One safe approach is to move `right` to the end first, then step it back to the last real character before the first comparison.

- Look at the comparison operator inside the `if` statement. When should the function return 0 — when the characters match or when they do not match?
-

Question 9 - Broken Code: `str_starts_with_p` [10 points]

The function below is supposed to check whether string `str` starts with the given `prefix`. It should return 1 if `str` starts with `prefix` and 0 otherwise. It has **two bugs** — both return values are swapped.

```
int str_starts_with_p(char *str, char *prefix) {
    while( *prefix ) {
        if( *str != *prefix ) return 1;
        str++;
        prefix++;
    }
    return 0;
}
```

Fix `str_starts_with_p` and test it inside `broken_str_starts_with_p` with:

```
printf( "Hello World starts with Hello? %d\n", str_starts_with_p( "Hello World", "Hello" ) );
printf( "Hello World starts with World? %d\n", str_starts_with_p( "Hello World", "World" ) );
printf( "abc starts with abcdef? %d\n", str_starts_with_p( "abc", "abcdef" ) );
```

Expected output after fix:

```
Hello World starts with Hello? 1
Hello World starts with World? 0
abc starts with abcdef? 0
```

Hint:

- When a mismatch is found, should the function return 1 (yes) or 0 (no)?
 - When the entire prefix has been matched successfully, should the function return 1 or 0?
 - The third test checks a prefix that is longer than the string. If `str` reaches `'\0'` while `prefix` still has characters, that is a mismatch.
-

Question 10 - Broken Code: `last_char_p` [10 points]

The function below is supposed to return a pointer to the **last** occurrence of character `c` in string `s` by searching from the end. If the character is not found, it should return 0 (NULL). It has **two bugs**.

```
char *last_char_p(char *s, char c) {
    char *p = s;
    while( *p ) p++;
    while( p >= s ) {
        if( *p == c ) return p;
        p++;
    }
    return 0;
}
```

Fix `last_char_p` and test it inside `broken_last_char_p` with:

```
char str[20] = "banana";
char *result;

result = last_char_p( str, 'a' );
if( result ) printf( "Last 'a' at position %d\n", (int)(result - str) );
else printf( "'a' not found\n" );

result = last_char_p( str, 'z' );
if( result ) printf( "Last 'z' at position %d\n", (int)(result - str) );
else printf( "'z' not found\n" );
```

Expected output after fix:

```
Last 'a' at position 5
'z' not found
```

Hint:

- After the first loop, p points to '\0'. Should the backward search start from '\0' or from the last real character?
- In the second loop, is the pointer moving in the correct direction to search backwards?
- A safe backward-search pattern is: while p > s, move p one step back, then check *p.